

Week 6 • Routing, pages, and going live

Technical Interaction Design • ITU • Autumn 2026

Built 1 October 2026 • commit 7ac0b60

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-06.pdf>

CONTENTS

Sharing a List Is a Many-to-Many Relationship

- The line becomes a table: the join table `ListMember`
- Ada finds Armin by his exact username, if his row lets her
- Sharing writes the fact twice: a membership row, and the ACLs
- Armin's lists and Ada's members are ordinary queries
- Leaving is recorded in the data; access is still decided by the ACL
- Keeping `owner` next to `ListMember` is a choice you defend
- A join table, not a `Parse.Relation` or an array
- What came up in the lecture

Publishing Your App on GitHub Pages

- The built app is a folder of plain files
- GitHub Pages hands out those files, from your repository, for free
- Vite has to know the app does not live at the root
- `gh-pages` publishes `dist/` to a branch of its own
- The Parse keys travel with the build, and that is fine
- Open it on your phone
- What came up in the lecture

Routing in React

- One page is getting crowded, and people expect URLs anyway
- React does not route; a library does
- Routing with `react-router-dom`
- Going live with routes
- Notes
- What came up in the lecture
- Read More

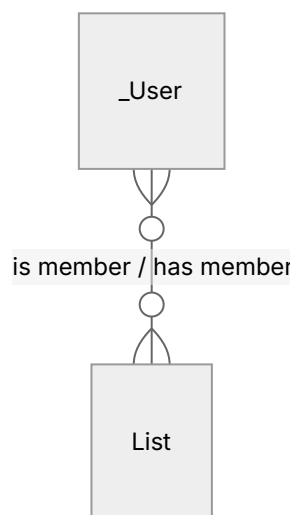
Sharing a List Is a Many-to-Many Relationship

Source: [Lectures/Technical/Backend/Many-to-Many-Sharing-a-List.md](#) — something unclear? [Open an issue](#), or send a pull request.

It follows on from [Relationships Between Object Classes](#), where a to-do got an owner and lists got to-dos, and from [Authentication and Authorization](#), where ACLs decided who may read what.

Ada is moving, and wants to share the *Apartment* list with Armin, who is helping her. A list can be shared with many users, and a user can have many lists shared with them. That is a **many-to-many** relationship. Every project in this course has one, because sharing at the list level is part of the required core.

On a whiteboard it is one line, with *many* at both ends. Read it left to right, a user is a member of many lists, and back, a list has many members:



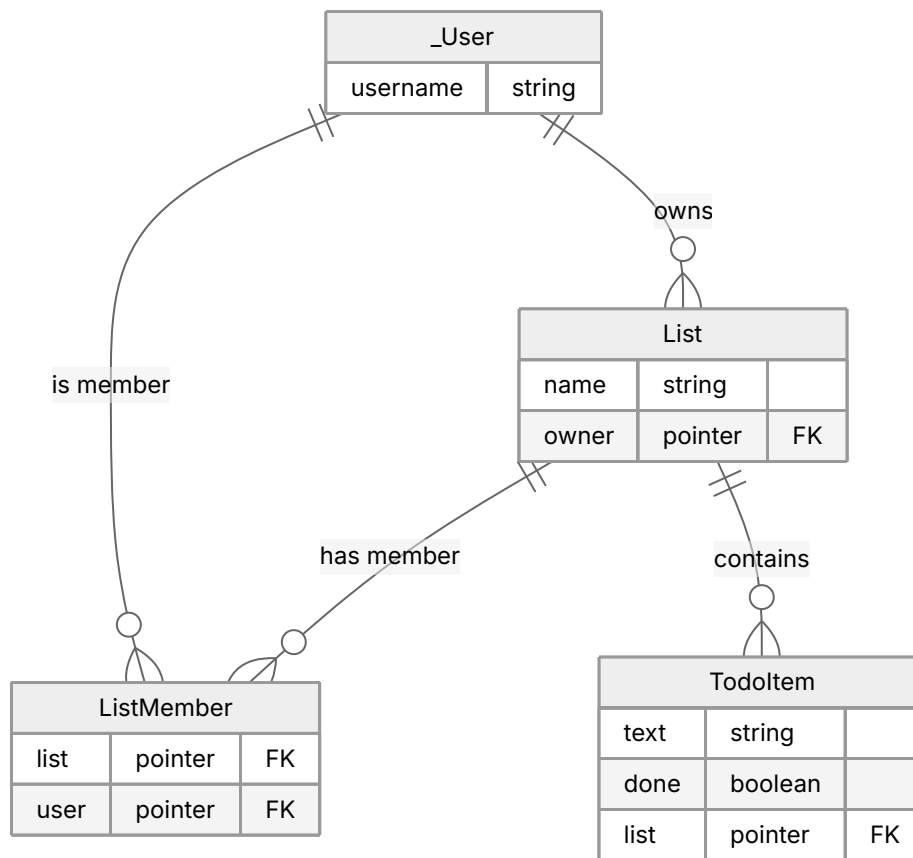
A database cannot store that line as it is. A pointer holds exactly one object, so it can only ever be the *one* end of a relationship. Where the line goes instead is the whole question of this note.

The ACL cannot be that place either. It can let Armin *read* the list, but it is a permission, not a relationship, as at the [start of the Relationships note](#): nobody can ask it “which lists are shared with me?”. And it belongs to the list, which is Ada’s. When the move is done and Armin wants *Apartment* off his screen, he cannot take himself off; with write access he could, but that is also the right to rename the list, delete it, or remove Ada.

So the membership gets a row of its own: one that Armin may delete, without being allowed to touch the list. A row also has room for anything you need to *store* about the membership: whether he may edit, an invitation he has to accept first.

The line becomes a table: the join table `ListMember`

Create a class whose job is to represent one membership: one row says *this user is on this list*.



The line from the whiteboard has become a table. Two pointers, one per side. That is all a join table is: two one-to-many relationships, meeting in the middle.

You will also meet it as a *junction*, *association* or *mapping* table, named after its two sides: `UserListMap`, or `lists_users` in Rails. That name says how the table works. Once it stores something about the relationship, like whether this member may edit, name it after what one row *is*: a membership. "Armin left the list" deletes a `ListMember`; nobody says they deleted a mapping.

Create the class in the dashboard first, the way you created `List`: a `ListMember` class with two Pointer columns, `list` to `List` and `user` to `_User`. Whether the client may create a class by saving to it depends on how your app is set up. Creating it by hand means you never have to find out in front of anybody.

Something unclear in this section? Open an issue.

Ada finds Armin by his exact username, if his row lets her

Sharing needs Armin as a `Parse.User`. The smallest way to get him: one text input where Ada types his username, exactly.

```
const friend = await new Parse.Query(Parse.User)
    .equalTo("username", name)
    .first();           // undefined if nobody has that username

if (!friend) {
    setError("No user with that name");
    return;
}

await shareList(list, friend);
```

No search, no autocomplete: less code, and better privacy, because Ada can only find someone whose username she already knows.

For the lookup to find Armin, three settings on `_User` have to agree.

A new user's row is private, so sign-up makes it readable

Out of the box, a user's row is readable only by that user, and the lookup finds nobody. Sign-up makes it readable:

```
await user.signUp();

const acl = new Parse.ACL(user); // I may change my row
acl.setPublicReadAccess(true);   // others may find me
user.setACL(acl);
await user.save();
```

That only helps users who sign up from now on. If your database already has users, their rows are still private, and sharing with them fails with *No user with that name*, although they exist. Tick **Public Read** on each of them, by hand, in the dashboard:

Edit Access Control List (ACL)

Public	☒ Read ☐ Write
Z7fVy29uEH	☒ Read ☒ Write
Add Role	Role or User...

[Learn more about ACLs and app security](#)

Cancel
Save ACL

The class lets only logged-in users ask

An ACL cannot say “logged-in users”, so the class-level permissions do. **Public** keeps only **Create**, because signing up is creating a user, and whoever signs up is not logged in yet. **Authenticated** gets everything except *Add field*.

That does not let Armin change or delete Ada’s account. The class only says who may *try*; each row’s ACL says who may read or write it, and a user’s row names only that user as a writer. Leave the class closed instead, and nobody could save even their own row, which sign-up does above.

Edit Class Level Permissions

Simple

Advanced

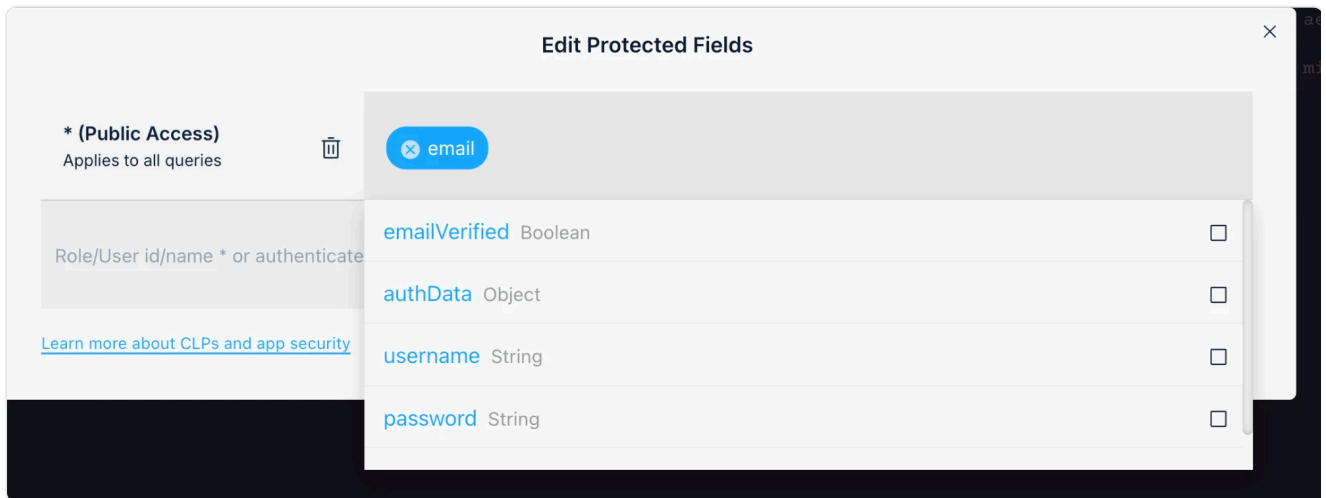
	Read	Write	Add
Public	☐ Get ☐ Find ☐ Count ☒ Create ☐ Update ☐ Delete ☐ Add field		
Authenticated	☒ Get ☒ Find ☒ Count ☒ Create ☒ Update ☒ Delete ☐ Add field		
Add Role	Role, User, or Pointer...		

[Learn more about CLPs and app security](#)

Cancel
Save CLP

Protected fields hide every column except the username

Protected Fields hides `email` by default. Add every column you put on `_User` yourself. Never add `username` : the lookup would stop working.



Anything more public about a person gets a `Profile` row

If people in your app need more public than a username (a display name, an avatar), give that part a row of its own: a public `Profile` , pointing at the private `_User` .

Something unclear in this section? Open an issue.

Sharing writes the fact twice: a membership row, and the ACLs

The row is the data: it says Armin is on the list. It has an ACL of its own, which lets **both** of them change it. But a row opens nothing. What lets Armin read the list is still the ACL, on the list *and* on each of its to-dos, because [Parse does not pass an ACL down](#):

```
const ListMember = Parse.Object.extend("ListMember");

export const shareList = async (list, friend) => {

  // The DATA: a row that says Armin is on the list
  const member = new ListMember();
  member.set("list", list);
  member.set("user", friend);
  const memberAcl = new Parse.ACL(Parse.User.current()); // Ada
  memberAcl.setReadAccess(friend, true); // and Armin
  memberAcl.setWriteAccess(friend, true);
  member.setACL(memberAcl);

  // The SECURITY: Armin may read the list, and every to-do in it
  const acl = list.getACL();
  acl.setReadAccess(friend, true); // may read, not change
  list.setACL(acl);
  const todos = await new Parse.Query(TodoItem).equalTo("list", list).find();
  todos.forEach((todo) => todo.setACL(acl));

  await Parse.Object.saveAll([member, list, ...todos]); // one request, not a transaction
};
```

Something unclear in this section? Open an issue.

Armin's lists and Ada's members are ordinary queries

The home page asks for two things: the lists I own (`owner` is me), and the lists I am a member of. It says what the screen is for, so a list someone forgot to give an ACL does not turn up on everybody's home page. And leaving is one line, which Armin is allowed to run:

```
// Armin: the lists shared with me
const shared = new Parse.Query(ListMember);
shared.equalTo("user", Parse.User.current());
shared.include("list");
shared.include("list.owner"); // through the list, to its owner: "shared by ada"
const memberships = await shared.find();

// Ada: who is on this list?
const members = new Parse.Query(ListMember);
members.equalTo("list", apartmentList);
members.include("user");
const people = await members.find();

// Armin leaves
await membership.destroy();
```

The same shape fits any many-to-many: labels on to-dos would be a `TodoLabel` class, with a `todo` and a `label` pointer.

In [todo-26](#): `git checkout week-05-sharing` ([browse it](#)). Look at `App.jsx`: `handleShare` writes both copies of the fact, the ACLs on the list and its to-dos and the `ListMember` row, in one `saveAll`; `loadLists` asks for my lists and my memberships; `handleLeave` deletes my row. And in `AuthPage.jsx`, sign-up makes the new user's row readable, so others can find it.

Something unclear in this section? Open an issue.

Leaving is recorded in the data; access is still decided by the ACL

Armin has destroyed his membership, and the list is off his screen. But the ACLs on the list and its to-dos still name him, and the server still lets him read them. `ListMember` is data; it opens and closes nothing.

So the same fact now lives in two places: `ListMember` says who is on the list, the ACLs say who may read it. That is the redundancy argument from [the refactoring above](#). This time it cannot be removed, because one copy is the data and the other is the security. Sharing has the same problem the other way round. `saveAll` sends the membership and the ACLs in one request, but not as a transaction. If it fails halfway, they disagree.

Keeping the two in step needs code with authority over Ada's objects, even when Armin is the one acting. That is the server. An `afterDelete` on `ListMember` that takes Armin out of the ACLs is exactly what [Running Code Server-Side](#) is for. Until then, the client does its best, and you know where the gap is.

Something unclear in this section? [Open an issue](#).

Keeping `owner` next to `ListMember` is a choice you defend

Once `ListMember` exists, Ada could be a member of her own list too, and the `owner` pointer would go. Or `owner` stays, because *the one who may delete the list and share it* is a different thing from *a member who reads it*. Both are defensible; which one is right depends on what your users do. One thing tips it: `owner` holds exactly one user, so only one person can ever delete the list. If your app needs several admins, the owner moves into `ListMember`, as a column that says what each member may do. It is exactly the kind of decision the data-model page of your report asks you to explain.

Something unclear in this section? [Open an issue](#).

A join table, not a `Parse.Relation` or an array

Do not model relationships with `Parse.Relation`

You will meet `Parse.Relation` in the documentation, which is Parse's built-in way of doing many-to-many. It is less typing, and it cannot carry any information about the relationship, or have an ACL of its own that lets Armin leave. So we are not going to use it. Knowing that it exists is enough.

Do not model relationships with arrays

Parse lets you store an array of objects in a field, so a `members` array on the list is tempting. Resist it. An array has no room for information about the relationship. It lives inside the list, so only the list's owner can change it. It has to be rewritten in full to add one element. And it gets slow and awkward as soon as it is not tiny. Pointers for one-to-many, a join table for many-to-many. Those two cover everything you need this semester.

Something unclear in this section? [Open an issue](#).

What came up in the lecture

Things that happened while this was coded live, rather than things that were planned.

Typing a username is the simplest way to share, not the only one

Ada could also send Armin a link to the list. Opening it asks to join, and Ada gets a request she can accept. A link also fits in a QR code, which is how WhatsApp adds a contact: one person shows their code, the other scans it, and lands on the URL that creates the relationship. Both need a URL per list, which is what [Routing in React](#) gives you. Typing the username is less code, and enough to start with.

An invitation deserves a class of its own

If Armin has to accept before he is on the list, the pending state could be a column on `ListMember`, and every query for memberships would have to leave the pending ones out. Or it gets a class of its own: sharing creates an `Invitation`, Armin sees his invitations when he logs in, and accepting one creates the `ListMember` row. Both work. The second gives the concept a name, and a design whose concepts have names is easier to read, for you and for anyone, or any agent, working on your code.

Checking out a tag leaves you on no branch, and `git stash` lets you move on

`git checkout week-05-sharing` checks out a tag: one commit, not a branch. Git warns that you are in *detached HEAD* state. It looks scarier than it is. A branch moves forward with every commit you make; a detached HEAD is a snapshot you look at. For reading and running the code from the lecture, that is all you need.

Change a file, then try to check out the next tag, and git refuses: your local changes would be overwritten. It is protecting work you do not care about. `git stash` puts the changes aside, on a stack you can take them back from with `git stash pop`, and the checkout goes through. `git checkout .` throws them away instead. If you do want to keep working on top of a tag, give the work a branch first: `git switch -c my-experiment`.

Something unclear in this section? [Open an issue](#).

Exam Questions

1. Why is a join table preferred over an array field or a `Parse.Relation` for list membership?
2. Why is list membership a `ListMember` row, and not only an entry in the list's ACL? After Armin deletes his row, why can he still read the list?
3. Once `ListMember` exists, do you still need an `owner` pointer on `List`? Argue for one answer.

Publishing Your App on GitHub Pages

Source: [Lectures/Technical/Tooling/Publishing-on-GitHub-Pages.md](#) — something unclear? [Open an issue](#), or send a pull request.

Until today the app has lived on `localhost:5173`, on your laptop, where nobody else can reach it. This note puts it on the internet, at an address anyone can open on their phone. It takes a few minutes, and you do it once; after that, publishing again is one command.

The built app is a folder of plain files

`npm run dev` is for working: it rebuilds on every save. What you publish is the result of

```
npm run build
```

which writes a folder `dist/`: an `index.html`, and next to it the JavaScript and CSS that Vite bundled.

No server code at all. Your backend is Parse, and it has been on the internet since week 4; only the frontend was on your laptop. So anything that can hand out files can host the app. `npm run preview` serves `dist/` locally, if you want to see the built version first.

Something unclear in this section? [Open an issue](#).

GitHub Pages hands out those files, from your repository, for free

Every public repository can have a site at `https://<owner>.github.io/<repo>/`. For your team, `<owner>` is the organisation. A free organisation gets Pages for **public** repositories only; a private one has its Pages settings greyed out.

Something unclear in this section? [Open an issue](#).

Vite has to know the app does not live at the root

The address ends in `/<repo>/`, not in `/`. By default Vite writes `/assets/index-....js` into `index.html`, and on GitHub Pages that path is wrong: the page loads, finds none of its scripts, and stays blank. Tell Vite, in `vite.config.js`:

```
export default defineConfig({
  plugins: [react()],
  base: "<repo>/", // e.g. "/todo-26/"
})
```

The dev server moves with it: from now on it is `localhost:5173/<repo>/`.

Something unclear in this section? Open an issue.

gh-pages publishes `dist/` to a branch of its own

Install the small tool that does the copying, as a development dependency:

```
npm install --save-dev gh-pages
```

Add a script to `package.json`:

```
"deploy": "npm run build && gh-pages -d dist"
```

and run it:

```
npm run deploy
```

It builds, then pushes the contents of `dist/` to a branch called `gh-pages`, separate from your code. The first time, tell GitHub to serve that branch, in the **repository's** settings (the organisation has a *Settings* tab and a *Pages* page too, but that one is only about domains): **Settings** → **Pages** → **Build and deployment** → **Deploy from a branch** → **gh-pages**, / (root). The first deployment takes a minute or two; the Pages settings show the address when it is live.

From then on, every change goes out with `npm run deploy`.

Something unclear in this section? Open an issue.

The Parse keys travel with the build, and that is fine

`.env.local` is git-ignored, so the keys are not in your repository. But the build reads them, and bakes them into the JavaScript it publishes. That is not a leak: the same keys were already in the browser of everybody who opened your app, as the `curl` in [Authentication and Authorization](#) showed. What protects your data is the ACLs and the class-level permissions, not the keys.

It does mean the build has to run where `.env.local` is, which is on your laptop. A teammate who deploys needs their own copy.

Something unclear in this section? [Open an issue](#).

Open it on your phone

Log in, and your lists are there: same backend, same data, a different device. From now on two people testing sharing can be two people, on two phones, and not two windows on one laptop.

In [todo-26](#): `git checkout week-06-deploy` ([browse it](#)). Look at `base` in `vite.config.js`, and the `deploy` script in `package.json`.

Once the app has routes, publishing needs two more things: the router has to know about `/<repo>/` too, and a refresh on any page but the first answers 404. Both are in [Routing in React](#).

Something unclear in this section? [Open an issue](#).

What came up in the lecture

Things that happened while this was coded live, rather than things that were planned.

The database stays where it was; the page on GitHub Pages talks to it

Publishing moves only the frontend. On `localhost`, the app sent its requests to Parse; now the same JavaScript, loaded from `github.io`, sends the same requests to the same server, with the same keys. That is why your lists are there on the phone. It also means the app on your laptop and the published one share one database: what you try out locally, your published users see.

Something unclear in this section? [Open an issue](#).

Exam Questions

1. Your app is built and published, and the page is blank. The browser's Network tab shows the JavaScript file failing with 404. What did you forget, and why does it matter on GitHub Pages?
2. The Parse keys are in a git-ignored `.env.local`, and yet they are in the published app. Is that a problem? What protects your data instead?

Routing in React

Source: [Lectures/Technical/Routing/Routing-in-React.md](#) — something unclear? [Open an issue](#), or send a pull request.

One page is getting crowded, and people expect URLs anyway

Our app shows every list on one page, and it is getting crowded. What people want is a page per list, and they want each page to have its own address:

- **a link** Ada can send Armin, that opens *Apartment* and nothing else;
- **a bookmark** that comes back to the same list tomorrow;
- **the back button**, taking them to where they just were.

The browser still loads one `index.html`. So the app itself has to notice every change of address, and draw the page that belongs to it. That is client-side routing.

Something unclear in this section? [Open an issue](#).

React does not route; a library does

React draws components, and has no idea what a URL is. Routing comes from a library that sits between the address bar and your components. When somebody clicks a link or presses *back*, it catches the navigation before the browser can ask the server for a new page, changes the address itself, and tells React which page to draw.

Which library? Look on npm and take the one everybody uses. Popularity is not a matter of taste here: it buys documentation, answers to the question you are about to search for, and many eyes on its bugs. For React, that is `react-router-dom`:

```
npm install react-router-dom
```

Something unclear in this section? [Open an issue](#).

Routing with `react-router-dom`

Everything below is from `todo-26`, or would fit into it.

BrowserRouter wraps the app once, in **main.jsx**

```
import { BrowserRouter } from "react-router-dom";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter basename={import.meta.env.BASE_URL}>
      <App />
    </BrowserRouter>
  </StrictMode>,
);
```

It watches the address bar, and every component inside it can ask what the URL is. (**basename** is for [going live](#); leave it out and nothing changes on **localhost** .)

Routes picks one Route per URL

At the end of `App`, after the login check, the app no longer returns one page. What it used to return, the lists and the forms, moves into its own component, `Home`, in `Home.jsx`. `App` keeps the state and the handlers, and passes them down as props. Then it returns the list of pages, each with the path it answers to:

```
return (  
  <Routes>  
    <Route  
      path="/"   
      element={   
        <Home  
          lists={lists}  
          memberships={memberships}  
          onAddList={handleAddList}  
          onShare={handleShare}  
          onLeave={handleLeave}  
          onLogout={handleLogout}  
        />  
      }  
    />  
    <Route path="/lists/:listId" element={<ListPage />} />  
    <Route path="*" element={<Link to="/">No such page</Link>} />  
  </Routes>  
);
```

`element` is the JSX to draw for that path. `Home` gets everything it shows and everything it can do from `App`; `ListPage` needs nothing, because it reads its list from the URL. The `*` route catches every URL nothing else matched: a mistyped link gets an answer instead of an empty page.

A path matches the whole URL. `/lists/:listId` matches `/lists/Xk3v9QaB2c`, and not `/lists/Xk3v9QaB2c/edit`. (Older tutorials write `exact` and `component=`. That is React Router 5; since version 6 neither exists, and matching is exact by default.)

`:listId` is a parameter, and `useParams` reads it

The part of the path after a colon is a placeholder. On `/lists/Xk3v9QaB2c`, `useParams()` gives `{ listId: "Xk3v9QaB2c" }`: the `objectId` of the list, straight from the database into the address bar. The page loads its own list:

```
export default function ListPage() {
  const { listId } = useParams();

  const [list, setList] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadList() {
      try {
        setList(await new Parse.Query(List).get(listId));
      } catch (err) {
        // "Object not found": no such list, or its ACL does not let me read it
        setError(err.message);
      }
    }
    loadList();
  }, [listId]);

  return (
    <
      <Link to="/">< All lists</Link>
      {error ? <p>{error}</p> : !list ? <p>Loading...</p> : <ToDoList list={list} />}
    </>
  );
}
```

The three states of remote data again: loading, error, the list. The error state matters more than it looks. Send somebody the link to a list that is not shared with them, and `get()` fails with *Object not found*. **A URL is not a permission**: anyone can type any address, and the ACL still decides what comes back.

In todo-26: `git checkout week-06-routing` ([browse it](#)). Look at `main.jsx` (`BrowserRouter`, with a `basename`, explained under *Going live with routes*), `App.jsx` (three routes, and the state and handlers it passes to `Home`), the new `Home.jsx` (lists by name, each a `Link` to `/lists/:listId`), and `ListPage.jsx`: `useParams` gives the `listId`, and `new Parse.Query(List).get(listId)` loads it. Open a list that is not yours, and the page shows the server's *Object not found*. A URL is not a permission; the ACL decides.

Link changes the URL without asking the server; **<a>** reloads everything

On the home page, every list name is a link to its page:

```
<Link to={` /lists/${item.id}`}>{item.get("name")}</Link>
```

It renders as an ordinary **<a>**, but a click changes the URL in the browser, and the router draws the new page. Nothing is fetched but the list. Write **** instead, and the browser asks the server for a new page: the whole app starts again from **index.html**, with every piece of state gone.

useNavigate changes the URL from code

A link is for the user to click. When the *code* decides where to go, for instance to open a list right after creating it, ask for **navigate**:

```
const navigate = useNavigate();

async function handleAddList(name) {
  // ... create and save the list, as before ...
  const savedList = await list.save();
  navigate(`/lists/${savedList.id}`); // straight to the new list's page
}
```

(Not in **todo-26**. A good first thing to try on your own!)

Nested routes share a layout, and `<Outlet>` marks where the child goes

Every page will want the same frame: the app's name, a link back to *My lists*, *Logout*. Instead of repeating it in each page, give the routes a parent with no path of its own:

```
<Routes>
  <Route element={<Layout />}>
    <Route path="/" element={home} />
    <Route path="/lists/:listId" element={<ListPage />} />
  </Route>
</Routes>
```

```
function Layout() {
  return (
    <div>
      <header>
        <Link to="/">My lists</Link>
      </header>
      %%render your sidebar %%

      <Outlet />    {/* the matched child route renders here */}

    </div>
  );
}
```

(Not in `todo-26` either. Worth doing the day you have a third page.)

Search parameters, like `?show=open&style=minimal`, are read with `useSearchParams`

Some state belongs in the URL, so that a link carries it: which to-dos a list page shows, say.

`/lists/Xk3v9QaB2c?show=open` shows only the open ones:

```
const [searchParams, setSearchParams] = useSearchParams();
const onlyOpen = searchParams.get("show") === "open";
const visible = onlyOpen ? todos.filter((todo) => !todo.done) : todos;

// ...
<button onClick={() => setSearchParams({ show: "open" })}>Only open</button>
```

Pages only for logged-in users are a note of their own

`todo-26` still shows the login page to anybody who is not logged in, with the early return from week 5, before any route is drawn. Doing it per route is [Protecting Routes If Not Logged In](#).

Something unclear in this section? Open an issue.

Going live with routes

Publishing the app is in [Publishing Your App on GitHub Pages](#). Routes add two things to it.

The router has to know the app lives under `<repo>/` too

`base` in `vite.config.js` told Vite; `basename` tells the router, so that `/lists/abc` in your routes means `<repo>/lists/abc` in the address bar:

```
<BrowserRouter basename={import.meta.env.BASE_URL}>
```

`import.meta.env.BASE_URL` is that same `base`, so there is only one place to change it.

A refresh asks the server, and GitHub Pages answers 404

Click from the home page to a list, and it works. Refresh, and GitHub answers with its 404 page.

The refresh asked the *server* for `<repo>/lists/Xk3v9QaB2c`. The server has one page, `index.html`. The list page never existed anywhere but in the browser, where the router drew it. That is what client-side routing *is*, and you only really believe it once it has bitten you. Opening a link to a list from a chat message fails the same way.

The fix: make every unknown path load the app anyway, so the router gets to read the URL. Many hosts have a setting for that. GitHub Pages does not, but for any path it does not know it serves `404.html`. So the build copies `index.html` there:

```
"build": "vite build && node -e \"require('fs').copyFileSync('dist/index.html', 'dist/404.html')\""
```

(`node` rather than `cp`, so that it works on Windows too.) The response still carries the status 404. Browsers do not care; a search engine would. The alternative is `HashRouter`: the URLs become `<repo>/#/lists/Xk3v9QaB2c`, and the server never sees the part after the `#`. It works, and the URLs are uglier.

In **todo-26**: `git checkout week-06` ([browse it](#)). Look at the `build` script in `package.json`. This is the app as it stood at the end of the lecture.

Something unclear in this section? Open an issue.

Notes

Other routers solve the same problems

- if you understand the concepts here you will have a much easier time understanding other similar libraries
- the problems described above are the same

Something unclear in this section? Open an issue.

What came up in the lecture

Things that happened while this was coded live, rather than things that were planned.

Keeping the current page in `useState` means writing your own router

A `page` state variable and conditional rendering would switch pages, and it is how a router starts. But `useState` knows nothing about the address bar. The back button, a link that opens one list, a refresh that lands where you were: each would have to be wired by hand. `react-router-dom` has already done that, so we take it.

Something unclear in this section? Open an issue.

Read More

- [React Router Declarative Mode](#) - the official documentation - note that there's also Data Mode and Framework mode that we didn't talk about

Exam Questions

1. Why does a single-page app need routing in the browser, when a classic website does not?
2. What is the difference between `<Link>` and `<a>` in React Router? What happens to the app's state with each?
3. Explain what this routing setup does, for the URLs `/`, `/lists/abc`, and `/lists/abc/edit`:

```
<Routes>
  <Route element={<Layout />}>
    <Route path="/" element={home} />
    <Route path="/lists/:listId" element={<ListPage />} />
  </Route>
  <Route path="*" element={<NotFound />} />
</Routes>
```

4. On `/lists/abc`, how does `ListPage` find out which list to show? And what does it show when the list exists, but is not shared with you?

5. How would you read `show` from `/lists/abc?show=open`, and why keep it in the URL rather than in `useState`?

6. Your app is on GitHub Pages. Clicking to `/lists/abc` works; refreshing that page gives a 404. Why, and how do you fix it?

Something unclear in this section? Open an issue.